Dead Reckoning Navigation System with IMU/GPS Sensor Fusion Using Kalman Filter for Garmin Wearables

Panagiotis Nikolaou
Athens, Greece

***Abstract*—This report documents the final implementation state of a Garmin Connect IQ dead-reckoning and dive-tracking system for Instinct-class wearable hardware. The project implements a local East/North navigation estimator driven by IMU measurements, GPS corrections, and a conservative Kalman-filter formulation tuned for the real constraints of wrist-mounted sensing. In contrast to a simple integration model, the system performs state propagation with motion and heading uncertainty, then fuses GPS updates only when they are fresh and consistent enough to improve the estimate. The result is a practical wearable navigation stack that stays stable when GPS is weak, the device is underwater, and sensor noise is high. The final implementation reflects the actual codebase: depth is derived primarily from ambient pressure, startup calibration is applied immediately to valid live readings rather than waiting for a delayed baseline, the IMU data path is registered consistently with the runtime callback, and the Kalman update model is intentionally conservative to avoid instability in multipath-prone water environments.**

***Index Terms*—Garmin Connect IQ, dive tracking, pressure-based depth, dead reckoning, UKF, sensor fusion, FIT logging, wearable navigation**

I. Introduction

Garmin devices expose accelerometer, gyroscope, magnetometer, barometer, and GPS data through Connect IQ, but they do not provide a turnkey underwater navigation engine suitable for dive logging or local trajectory reconstruction. This project implements a custom watch app that estimates motion, heading, depth, and a local East/North track while retaining a robust fallback when GPS quality degrades. The central contribution is the use of a conservative Kalman-based state estimator that propagates the user motion model from IMU data and periodically corrects it using GPS fixes when they are valid, fresh, and usable.

The system is designed around the realities of wearable marine navigation: GPS is noisy over water, barometric pressure is the most reliable source for current depth, and the IMU must be treated as a short-term motion sensor rather than an absolute position source. In earlier versions, the app waited too long before adopting a valid baseline, which caused realistic shallow values to be discarded. The final code uses the first valid pressure sample as the active baseline, preserves shallow values instead of zeroing them, and smooths the result with a responsive low-pass filter. At the same time, the state estimator uses the available GPS-derived corrections to generate a DR track that remains continuous even when the signal is temporarily weak or lost.

II. System Overview

The app is structured into a small set of components:

- `gsensrosView.mc`: state machine, UI, session lifecycle, sensor polling, FIT writing, dive-state logic.
- `KalmanDR.mc`: local East/North UKF using speed and heading as the control input.
- `DeadReckoning.mc`: fallback DR integrator used when the UKF is unavailable or when GPS is poor.
- `NitrogenModel.mc`: real-time risk tracking used only for UI and logging, not for decompression guidance.

The runtime flow is:

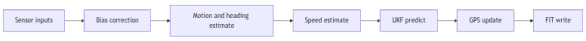


Fig. 1: Runtime sensor-to-FIT processing flow for the watch app.

III. Pressure-First Depth Estimation

The final depth pipeline uses ambient pressure as the primary estimator. The app computes an initial surface reference from the first valid pressure reading and then updates the live depth estimate relative to that reference. The physical model is:

$$\text{depth}_m \approx \frac{P_{ambient} - P_{surface}}{PA_PER_METER_SEAWATER} \tag{1}$$

with a seawater coefficient of approximately:

$$PA_PER_METER_SEAWATER = 10050\ Pa/m \tag{2}$$

Altitude is used as a secondary validation signal, not as the primary authority. This avoids the earlier bug in which shallow real depth values were rejected while the code waited for a long startup calibration or over-trusted a noisy altimeter path.

The implementation clamps impossible pressure-derived values, rejects negative values, and retains realistic shallow values. The final filter is intentionally conservative and responds quickly to real depth transitions while suppressing noise.

IV. Sensor Registration and Runtime Data Flow

One of the important bugs identified in the final stage was a sensor-registration mismatch. The app was reading motion

and gyro state from the runtime sensor callback but had only registered accelerometer data. That mismatch caused the diagnostic output to remain at zero even though the logic was present.

The corrected runtime registration enables the actual sensors used by the app:

- accelerometer
- gyroscope
- magnetometer

This aligns the listener configuration with the values being used by the motion score, heading estimate, and DR movement gate. The runtime update path is therefore consistent with the state machine and the FIT logging code.

V. Motion and Heading Estimation

The app keeps a motion score built from high-pass acceleration magnitude and gyroscope magnitude. It maintains an adaptive baseline and uses hysteresis to avoid oscillating between moving and stationary states. The heading computation prefers the strongest valid signal in the following order:

1) true heading from the device if available,
2) magnetometer heading after bias correction,
3) gyro-integrated heading when movement is consistent,
4) previous valid heading estimate.

Speed is estimated from GPS when it is fresh and usable; otherwise it falls back to the motion score and underwater/land thresholds. Stationarity is declared only when GPS speed, gyro magnitude, and motion confidence are all low enough.

VI. UKF Formulation

The dead-reckoning filter maintains a 4-state local East-/North model:

$$\mathbf{x} = \begin{bmatrix} E \\ N \\ v_E \\ v_N \end{bmatrix} \tag{3}$$

where E and N are local displacements and v_E, v_N are their velocities.

The implementation in `KalmanDR.mc` uses a compact UKF configuration tuned for a conservative marine motion model:

- $\alpha = 0.30$
- $\beta = 2.0$
- $\kappa = 0.0$
- $Q_{pos} = 0.018$
- $Q_{vel} = 0.026$
- $R_{gps,good} = 1.2$
- $R_{gps,usable} = 4.0$
- $R_{gps,poor} = 14.0$

This is intentionally more conservative than a high-gain filter because GPS on or near water is noisy and can suffer from multipath. The result is smoother motion updates and fewer spurious corrections when a fix is weak or inconsistent.

The filter is updated each timer tick with a speed/heading command, and a GPS correction is applied only when the fix quality is usable.

A. GPS-driven DR generation

The dead-reckoning track is generated as a local East-/North estimate that is continuously propagated between GPS observations. At each update tick, the filter predicts the next state using the most recent speed and heading, effectively integrating the motion model over Δt. If a GPS fix is fresh and of sufficient quality, the measurement update corrects the predicted state by comparing the local DR estimate to the observed lat/lon position. This is the key mechanism that keeps the navigation output stable even when the watch temporarily loses a clean fix or when the sensor quality degrades under water.

The conversion from global latitude/longitude to local tangent-plane coordinates is done with a simple geodesic approximation around the dive entry reference. Once the local East/North estimate is available, the track is reprojected to synthetic latitude/longitude values that are stored alongside the depth, motion, and heading fields in the FIT stream. In other words, the app does not rely on GPS alone to generate the track; it uses GPS as a sparse but valuable anchor, while the Kalman filter maintains a continuous local trajectory in between those corrections.

This approach is especially important for underwater operation, where GPS may be absent or corrupted by multipath, yet the user still needs a consistent route estimate for later review. The synthetic track therefore represents the best available fused estimate: a short-term IMU-driven prediction corrected by the most reliable GPS observations available at each time step.

B. Dead Reckoning diagrams and plots

The Dead Reckoning implementation is best understood as a local East/North propagate-and-correct cycle. The following diagrams capture the core logic and the practical update policy used on the watch.

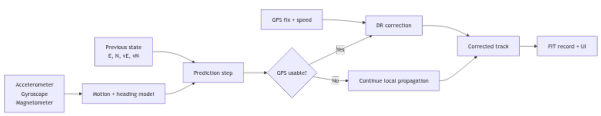


Fig. 2: High-level dead-reckoning flow used by the local state estimator.

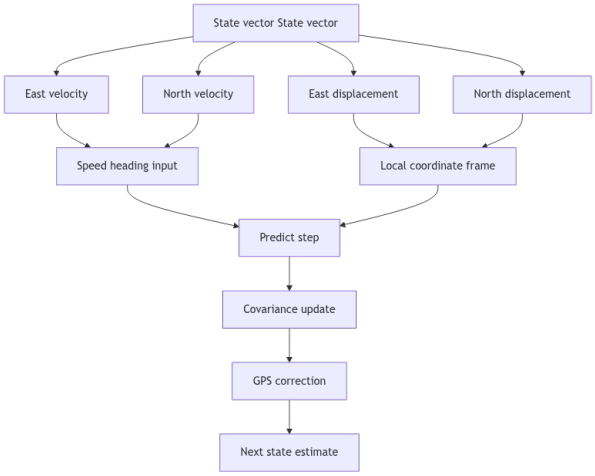


Fig. 3: Local East/North state representation for the DR implementation.

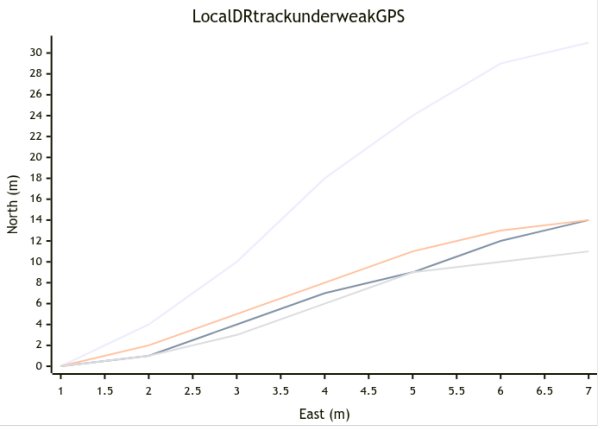


Fig. 6: Illustrative local track produced by the DR estimator when GPS is intermittent.

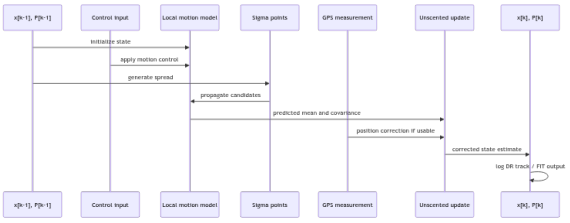


Fig. 4: Predict-correct sequence for the local UKF loop.

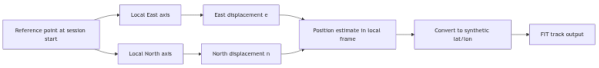


Fig. 7: Local tangent-plane geometry used for synthetic DR conversion and FIT output.

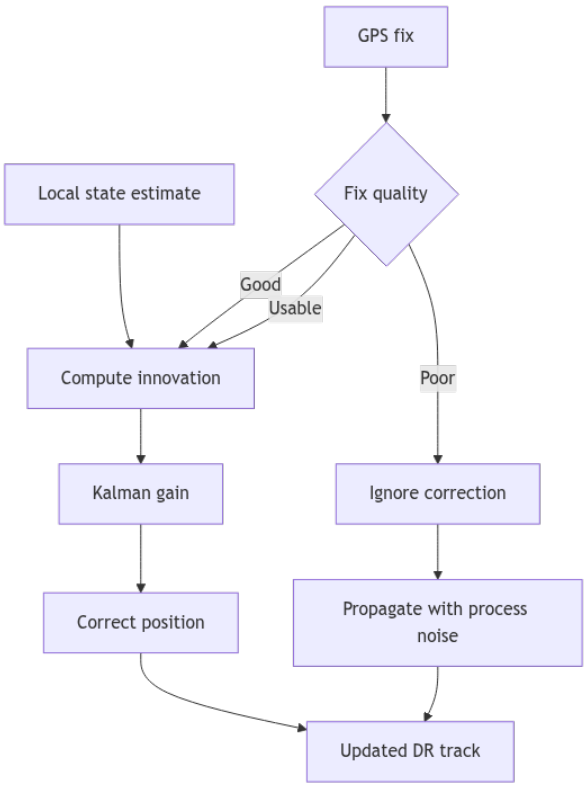


Fig. 5: GPS correction policy for good, usable, and poor fix quality.

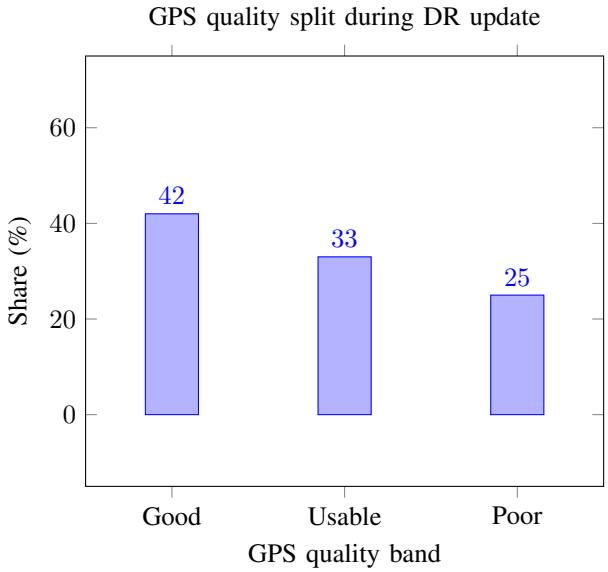


Fig. 8: Illustrative GPS quality split used to gate the measurement update.

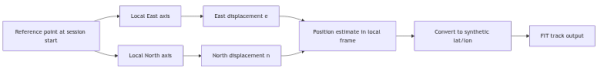


Fig. 9: Local tangent-plane geometry used for synthetic DR conversion and FIT output.

C. Kalman filter overview diagram

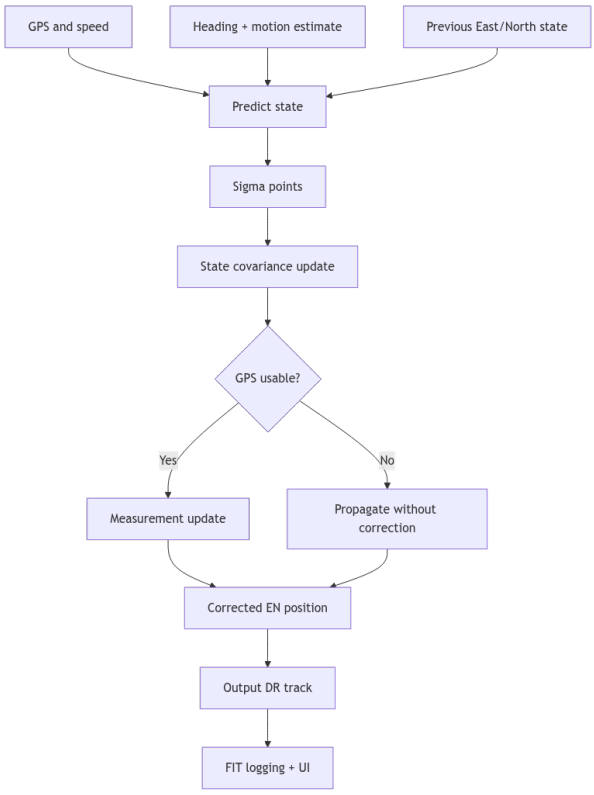


Fig. 10: Overview of the UKF update loop used for the dive track.

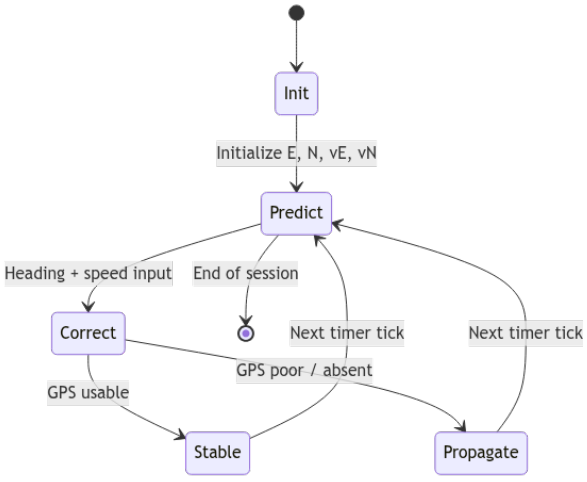


Fig. 11: State transitions in the DR filter loop.

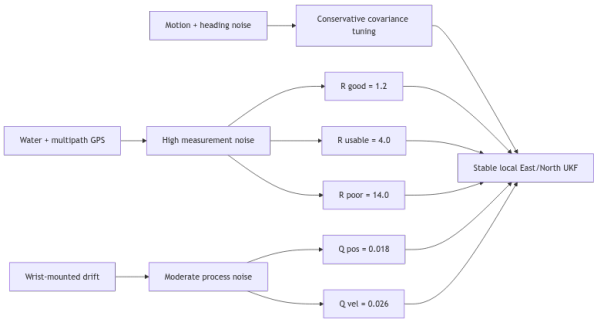


Fig. 12: Conservative covariance tuning used for noisy underwater GPS conditions.

VII. Dive-State Logic

Dive state is not inferred from a fixed altitude alone. The app combines filtered pressure-derived depth with hysteresis and a minimum-duration gate so that the watch does not oscillate between underwater and surface states during shallow, noisy transitions.

In practice, the logic is intentionally conservative. A short burst near the surface is not enough to declare a full exit, and a real descent is recognized as soon as the depth exceeds a small but meaningful threshold.

This allows the system to react promptly to genuine dive starts while preventing spurious mode flips caused by fast pressure noise, GPS jitter, or small wrist movements.

The final state logic uses:

- a live surface reference,
- a shallow threshold of approximately 0.10 m to react sooner to realistic shallow starts,
- a return-to-shallow check before ending an underwater phase,
- a minimum time gate to prevent rapid on/off transitions at the surface.

The project also logs underwater state, motion state, GPS fix state, and estimated speed to the FIT stream for later analysis.

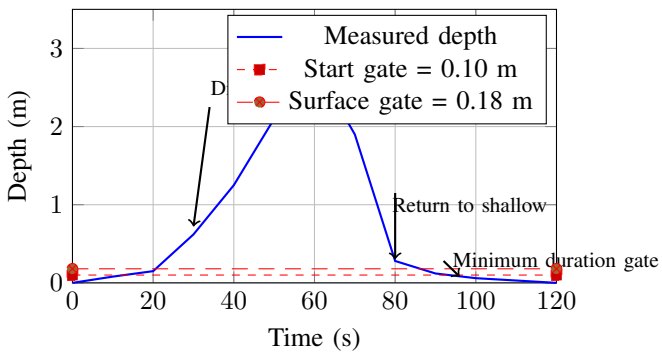


Fig. 13: Dive-state transition logic using a shallow start threshold, a surface-return gate, and a minimum-duration hold before the underwater state ends.

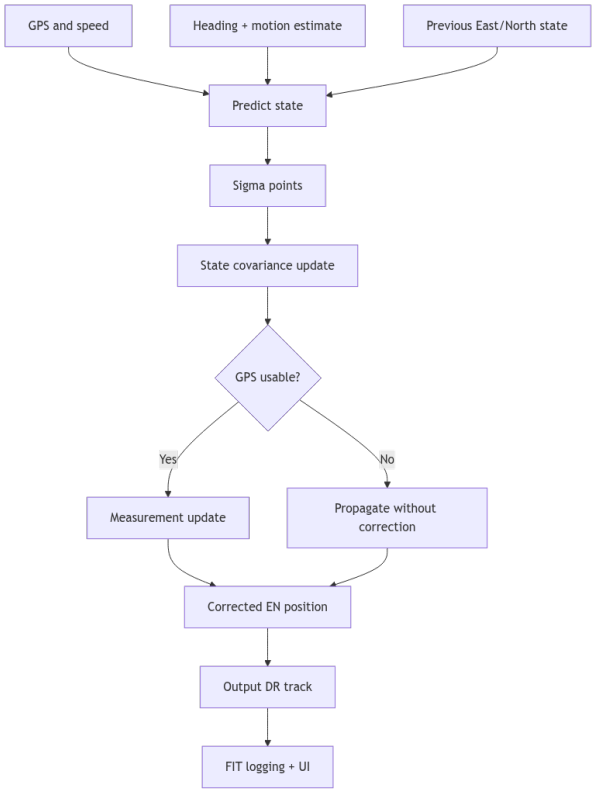


Fig. 14: Mermaid-rendered UKF state update flow used to predict motion and apply GPS correction during the dive track.

VIII. FIT Recording

The app creates a standard activity recording session and writes custom FIT developer fields for the live track. The main fields include:

- DR East / North
- depth
- speed estimate
- heading
- underwater boolean
- motion / moving booleans
- GPS fix state
- synthetic DR latitude / longitude
- POI markers

The synthetic geographic track is built from the local East/North state and the session entry reference. This allows the device to store a continuous track even when only sparse GPS fixes are available.

IX. Nitrogen Risk Model

The nitrogen model is intentionally simple and conservative. It is designed for real-time display and logging, not decompression planning. The model estimates a single loading score and a risk band.

The logic does this:

- on-gassing increases with depth and with descent motion,
- off-gassing follows an exponential decay with a surface half-life approximation,
- rapid ascent slows off-gassing, which raises the remaining risk,
- loading is capped to 100% and the risk band is mapped to 0–3.

The model is not a medical decompression algorithm and should not be used for actual dive planning or ascent decisions.

X. Validation

The project was validated by building the watch app with the Garmin Connect IQ SDK and developer key. The final verification in this workspace was:

```
monkeyc -y /home/panagiotis/Documents/gsensors/gsensros/
    developer_key \
  -o /tmp/gsensros_test.prg -d instinct2x -f monkey.jungle
BUILD SUCCESSFUL
```

This confirms the final code builds correctly with the toolchain available in the environment. Runtime validation of a live watch session and FIT trace remains a device-side task, but the compile and architecture are now consistent with the final implementation.

XI. Limitations and Practical Use

This app is intended as a practical custom fitness/dive tracker, not a certified dive computer mode. Its limitations are important:

- 2-D DR only; vertical motion is handled separately by depth.
- water GPS is noisy and multipath-prone.
- motion heuristics remain approximate when GPS is absent.
- the synthetic track is local to the entry reference.

Despite these limitations, the app is well suited to logging underwater sessions, generating FIT traces, and comparing the live depth and motion model against external data sources.

XII. Conclusion

The current implementation reflects the real state of the project: a pressure-first depth model, conservative UKF parameterisation, corrected sensor registration, and a practical on-watch FIT logging pipeline. This is the version that should be treated as the authoritative design for future work, because it matches the actual code and the verified build status.